

Systems programming

Lab 4 - Protocol buffers

Learning objectives

Use of protocol-buffers

Protocol buffers is a high level library for data serialization. This library can be use to allow communication of heterogeneous computers (different architectures or programming languages)

In this laboratory students will use the Protocol buffers library to implement the messages sent by ZerMQ sockets.

1 Protocol buffers

<https://protobuf.dev/overview/>

Protocol Buffers are a language-neutral, platform-neutral extensible mechanism for serializing structured data.

It's like JSON, except it's smaller and faster, and it generates native language bindings. You define how you want your data to be structured once, then you can use special generated source code to easily write and read your structured data to and from a variety of data streams and using a variety of languages.

Protocol buffers are a combination of the definition language (created in **.proto** files), the code that the proto compiler generates to interface with data, language-specific runtime libraries, and the serialization format for data that is written to a file (or sent across a network connection).

1.1 Installation

The standard protocol buffer packages are composed of a set of libraries and tools for automatic code generation. For standard languages (C++, C#, Java, Python, ...) only the official packages are required, while for the C programming language additional package are required.

Since there are are official packages for most of the Linux distributions and Mac OS X, it is possible to install the require tools and libraries using the common package managers:

Ubuntu

```
sudo apt update
sudo apt install protobuf-compiler
sudo apt install protobuf-c-compiler
sudo apt install libprotobuf-c-dev
```

MAC OS X

```
brew install protobuf
brew install protobuf-c
```

MAC OS X

In mac os X the correct way to install this library is running the previous brew commands. This will install all the necessary files, but, unfortunately they are not copied to the correct directories. In order to compile programs with protocol buffers follow the next steps

1 - Verify if the files are in the correct directory:

```
/opt/homebrew/Cellar/protobuf-c/1.5.0_14/
```

2 - If they are in the expected directory, specify those directories when compiling:

```
gcc application.c -o client-sub -lprotobuf-c
                        -L /opt/homebrew/Cellar/protobuf-c/1.5.0_14/lib
                        -I /opt/homebrew/Cellar/protobuf-c/1.5.0_14/include
```

If the package was installed in a different directory the command **brew info protobuf-c** will print that location

1.2 Programming model

Since the protocol buffers generates code to encode messages, besides the common steps related to the use of external libraries, it is necessary for the programmer to defined the message structures and compile those definitions into C code. The general programming workflow is as follows:

1. Definition of messages in the **.proto** file
2. Compilation of the **.proto** file to the required languages (**C** in our case)
3. Inclusion of the generated **.h** file into the code
4. Development of the code to pack/serialize and unpack/deserialize messages
5. Compile the application with the generated **.c** file and the **-lprotobuf-c** library

1.3 .proto file definition

The Protocol Buffers specification defines a syntax for the **.proto** files. Independently of the programming languages latter used, the syntax and semantic of these files is the same as explained in the official documentation:

<https://protobuf.dev/programming-guides/proto2/>

The **.proto** files allow programmers to define message type (+- like structures) and the fields that are contained in each message.

Each field has a type (scalar, strings, byte arrays or enumerates) and some optional labels as described in the documentation and summarized in the following tables:

<https://protobuf.dev/programming-guides/proto2/#scalar>
<https://protobuf.dev/programming-guides/proto2/#enum>
<https://protobuf.dev/programming-guides/proto2/#field-labels>

Filed value types	
double	
float	
int32	Inefficient for encoding negative numbers
int64	Inefficient for encoding negative numbers
uint32	
uint64	
sint32	Signed int value. These more efficiently encode negative numbers than regular int32s.
sint64	Signed int value. These more efficiently encode negative numbers than regular int64s.
fixed32	Always four bytes. More efficient than uint32 if values are often greater than 228.
fixed64	Always eight bytes. More efficient than uint64 if values are often greater than 256.
sfixed32	Always four bytes.
sfixed64	Always eight bytes.
bool	
string	A string must always contain UTF-8 encoded or 7-bit ASCII text, and cannot be longer than 2^{32}
bytes	May contain any arbitrary sequence of bytes no longer than 2^{32} .

Field Label	
optional	An optional field is in one of two possible states: - the field is set, and contains a value that was explicitly set or parsed from the wire. It will be serialized to the wire. - the field is unset, and will return the default value. It will not be serialized to the wire. You can check to see if the value was explicitly set.
repeated	this field type can be repeated zero or more times in a well-formed message. The order of the repeated values will be preserved.
required	When it is used, a well-formed message must have exactly one of this field.

The name of each field should be unique inside each message and the Field Numbers should:

- be a number between 1 and 536870911
- be unique among all fields for that message.
- not be changed once your message type is in use because it identifies the field in the message wire format.

1.4 .proto file compilation

After the definition of the message structure **.proto**, file it is necessary to compile this file to generate the required serialization code in the intended programming language.

For the officially supported languages (C++, C#, Java, Python, ...), the programmer should run the **protoc** command as explained in the tutorials:

<https://protobuf.dev/getting-started/>

For the generation of the C files it is necessary to run the **protoc-c** command:

```
protoc-c --c_out=. example.proto
```

this basic usage of **protoc-c** requires two arguments:

- **--c_out=OUT_DIR** - location of the resulting C files
- **.proto** file name

After the successful compilation, two new files are generated for each **.proto** file:

```
096 dez  5 11:00 client
310 dez  7 11:35 example.pb-c.c
572 dez  7 11:35 example.pb-c.h
865 nov 29 20:44 example.proto
987 dez  5 14:20 makefile
```

The name of the generated files depends on the original **.proto** file name.

Every time the **.proto** file is changes, it is necessary to regenerate the **.c** and **.h** files and recompile the whole project.

The folder **ProtobufExample** contains simple **protocol buffer** example. The makefile generates the C files necessary for processing the messages defined in the **example.proto** file.

After running make it is possible to observe and use the various generated data structures and functions in the files **example.pb-c.c** and **example.pb-c.h**. The following table shows the generated C structure for a sample message.

example.proto	example.pb-c.h
<pre>syntax = "proto2"; enum EnumType { ENUM_ONE = 0; ENUM_TWO = 1; } message Simple_message {</pre>	<pre>typedef enum _EnumType { ENUM_TYPE_ENUM_ONE = 0, ENUM_TYPE_ENUM_TWO = 1 } EnumType; struct SimpleMessage{ ProtobufCMessage base;</pre>

<pre> required string str_value = 1; optional int32 int_number = 2 [default = 10]; repeated float float_array = 3; optional EnumType enum_value = 4; } </pre>	<pre> char *str_value; protobuf_c_boolean has_int_number; int32_t int_number; size_t n_float_array; float *float_array; protobuf_c_boolean has_enum_value; EnumType enum_value; }; </pre>
--	--

The defined datatype is a simple C Structure that contains the following fields:

- values for scalars – example **int32_t int_number;**
- pointers for arrays – example **float *float_array;**
- number of elements for repeated fields – example **size_t n_float_array;**
- existence of value for optional values – example **protobuf_c_boolean has_int_number;**

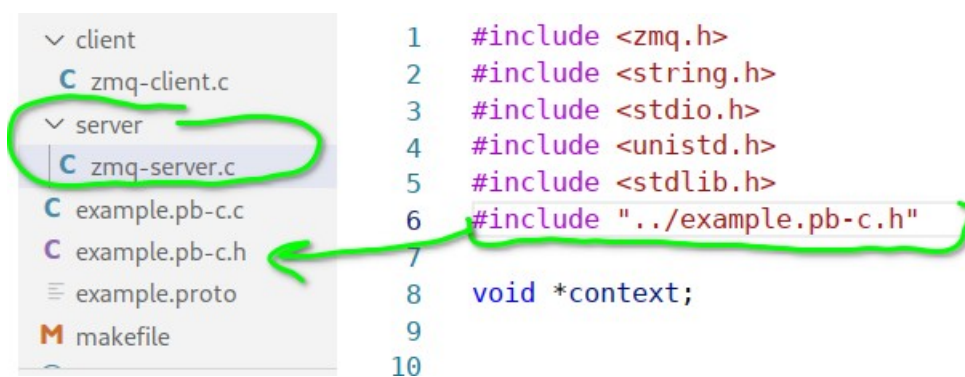
Along each message the protoc-c compiler also generates some auxiliary functions associated with each one:

<pre> #define SIMPLE_MESSAGE__INIT void simple_message__init (SimpleMessage *message) </pre>	Macro and function to initialize a simple_message structure
--	--

<pre>size_t simple_message__get_packed_size (const SimpleMessage *message)</pre>	Function to calculate how many bytes a message will occupy after serialized/packed. The returned value should be used to allocate the buffer that will contain the packed bytes.
<pre>size_t simple_message__pack (const SimpleMessage *message, uint8_t *out);</pre>	Function that receives the message structure, verifies if all required fields are set and serializes/packs the data. The packed/serialized bytes are stored in the buffer pointed by out. This buffers should have been previously allocated with the size returned by simple_message__get_packed_size
<pre>SimpleMessage * simple_message__unpack (ProtobufCAllocator *allocator, size_t len, const uint8_t *data);</pre>	This function receives as argument a buffer c(containing a previously serialized/packed message) and the length of such buffer and returns a message structure with the data in the buffer. If it is impossible to unpack the message null is returned. The allocator pointer should not be NULL.
<pre>void simple_message__free_unpacked (SimpleMessage *message, ProtobufCAllocator *allocator);</pre>	This function frees a message structure returned by the simple_message__unpack function. The allocator pointer should not be NULL.

Messages with different names will have different message structures and functions generated, all with an associated name.

The generated .h file should be included with every source code file that will process the defined messages, and the .c file should be compiled and linked along the final program, in the code and makefile



```
zmq-client: client/zmq-client.c example.pb-c.h
gcc client/zmq-client.c example.pb-c.c -g -o zmq-client -lzmq -lprotobuf-c

zmq-server: server/zmq-server.c example.pb-c.h
gcc server/zmq-server.c example.pb-c.c -g -o zmq-server -lzmq -lprotobuf-c
```

1.5 Development of the code to pack and unpack messages

With the help of the defined C message structures and functions, programmers can define the data to be serialized/packed and have it converted. The programmer will use variables of those a C structures to store the values to be serialized or retrieve the values just deserialized.

The general procedure for the use of protocol-buffers in a program requires:

- **for serializing/packing**
 - create a message C structure
 - fill the C message structure with the data values
 - pack the C message structure into a binary representation using the generated function
- **for serializing/unpacking**
 - store the binary representation of a message in a buffer
 - packing the buffer into the corresponding C message structure
 - access the data in the structure
 - destruct the message structure

The filling of the C message structures is done using regular C statements that manipulate the fields, depending on the types:

scalar	<pre>required int32 a=1; msg.a = 12;</pre>	The C message structure will contain a field with the suitable type and the name defined in the message
string	<pre>string d=1; msg.d = "xxxx";</pre>	The C message structure will contain a pointer to a string. The name of this pointer is the one defined in the .proto message
bytes	<pre>bytes name=1; msg.name.len= 10; msg.name.data = malloc(10); memcpy(msg.name.data, ptr, 10);</pre>	The C message structure will contain a variable with a pointer to the data and an integer with the length (in bytes) of the data. The data pointed by the data filed will be transferred unchanged.

optional	<pre>optional int32 b=2; msg.has_b = 1; msg.b = 13; msg.has_b = 0;</pre>	The C message structure, besides the field that will contain the data will contain another field that states whether the corresponding field is present.
repeated	<pre>repeated int32 c=1; msg.n_c = 200; msg.c = malloc (sizeof(int) * 200); // fill 200 int on msg.c</pre>	The C message structure will contain a pointer to an array of the specified values and the length of such array

The examples on the following page show how to define and access the various types of fields.

<https://github.com/protobuf-c/protobuf-c/wiki/Examples>

The example in folder **example-protobuf-c** shows the same procedures in messages with multiple fields

2 Protocol buffers and ZeroMQ and C

When using protocol buffers and sockets, it is necessary to read from the sockets packed messages, all of them with varying lengths. Since, for instance, arrays with different lengths will be packed into buffers also with different lengths, the programmer does not know what will be the length of the data to be read from the sockets. A solution such as the following may not work if the packed data may not fit into 100 bytes:

```
char msg_data[100];
int msg_len = zmq_recv(req_socket, buffer, 100, 0);
simple_msg = simple_message__unpack(NULL, msg_len, msg_data);
```

If the packed data (transmitted in the socket) is larger than 100 bytes, part of the message will be lost and the **simple_message__unpack** function will return an error.

To circumvent this, when reading messages from ZeroMQ it is fundamental to use an intermediary data structure: the **zmq_msg_t**, i.e. the ZeroMQ message type.

Information about management of **zmq_msg_t** can be accessed in the following pages:

https://libzmq.readthedocs.io/en/zeromq3-x/zmq_msg_init.html

https://libzmq.readthedocs.io/en/zeromq3-x/zmq_msg_recv.html

https://libzmq.readthedocs.io/en/zeromq3-x/zmq_msg_data.html

The previous code would become the following:

```
zmq_msg_t zmq_msg;  
zmq_msg_init (&zmq_msg);  
int msg_len = zmq_recvmmsg (responder, &zmq_msg, 0);  
void * msg_data = zmq_msg_data (& zmq_msg);  
simple_msg = simple_message__unpack(NULL, msg_len, msg_data);  
zmq_msg_close (zmq_msg)
```

A variable of type **zmq_msg_t** is created and initialized, and used to store the received message instead of a byte buffer. The programmer should read the message using the **zmq_recvmmsg**, and use the function **zmq_msg_data** to get the buffer of the data associated with the received message. After being processed the variable of type **zmq_msg_t** can be destroyed/closed.

This way, independently of the length of the data transmitted with **zmq_send**, the receiver will always get the data in a suitable sized buffer.

The code in **Protobuff_Example** folder also contains a two simple applications that exchange messages using rotocol buffers.

The code to serialize and send a messages is as follows:

```
SimpleMessage msg = SIMPLE_MESSAGE__INIT;  
msg.str_value = strdup(buffer);  
...  
int msg_len = simple_message__get_packed_size(&msg);  
char * msg_buf = malloc(msg_len);  
simple_message__pack(&msg, msg_buf);  
zmq_send (requester, msg_buf, msg_len, 0);
```

The code to receive and deserialize messages is the following:

```
zmq_msg_t zmq_msg;  
zmq_msg_init (&zmq_msg);  
  
int msg_len = zmq_recvmmsg (responder, &zmq_msg, 0);  
void * msg_data = zmq_msg_data (& zmq_msg);  
simple_msg = simple_message__unpack(NULL, msg_len, msg_data);  
printf("\tstring %s", simple_msg->str_value);  
...  

```

After calling the correct unpack function (**simple_message__unpack** in the previous example), it is possible to access the fields in the returned C structure to fetch the data sent by the server. This code does not include the destruction/free of the dynamically allocated memory.

The previous example only sends one type of message, but when receiving various types of messages, the programmer needs to know in advance the type of message that was read from the socket to call the correct unpacking function. ZeroMQ helps here by allowing the programmer to send a message identifier (with the **ZMQ_SNDMORE**) before sending the packed buffer:

Senders	Receiver
<pre>int msg_t = 1; zmq_send(sock1, &msg_t, sizeof(msg_t), ZMQ_SNDMORE); zmq_send (sock1, &msg, msg_len, 0);</pre>	<pre>int msg_t; zmq_recv (sock_2, &msg_type, sizeof(msg_type), 0); if(msg_t == 1){ zmq_recvmsg(sock, &msg_raw, 0);</pre>

If the protocol only accepts one specific message type, it is not necessary to send the message type as a part of the ZeroMQ message.

3 Exercise 1

The provided code (**Exercise_1**) is a solution to the exercise 4 of lab 3. In this exercise students should replace all the messages sent as strings by similar protocol buffer messages. The file **letter-movements.proto** already contains a possible definition of the messages to be sent to the server when the user moves a letter.

Student should update the provided **letter-movements.proto** with messages to all the interactions:

- client connects and sends a letter
- server confirms (or not) the client
- client sends movement to the server
- server confirms movement or hit the wall

Define in the file **balls.proto** the structure of the three missing messages, and update the functions in the **comm-fifo.c** file to replace the strings by the corresponding protocol buffer messages.

4 Exercise 2

Try to implement a new client in a different language (for instance **java** or **python**). This client will connect to the server and send random movements until it hits the wall.

In order to use protocol and ZeroMQ sockets students should also install the following:

- **for python**
 - pip install pyzmq
 - pip install protobuf
- **for java**
 - <https://github.com/protocolbuffers/protobuf/blob/main/java/README.md>
 - <https://github.com/zeromq/jeromq>

and follow the next tutorials

- **for python**
 - <https://zeromq.org/languages/python/>
 - <https://protobuf.dev/getting-started/pythontutorial/>
- **for java**
 - <https://zeromq.org/languages/java/>
 - <https://protobuf.dev/getting-started/javatutorial/>